

# AVR Embedded Systems – 2-Day Assignment

---

## Task 1: Implement `millis()` Using a Timer

**Objective:** Recreate the Arduino `millis()` function using one of the timers on the **ATmega328P**.

### Requirements:

- Configure a timer in **Normal Mode**.
- Use the **Overflow Interrupt** to count elapsed time.
- Calculate how many ticks/overflows represent 1 millisecond.
- Provide a function:

```
uint32_t millis(void);
```

### Hints:

- Datasheet sections:
  - Timer/Counter Normal Mode
  - Registers: `TCCRnA`, `TCCRnB`, `TCNTn`, `TIMSKn`
- Equation:

$$T_{\text{overflow}} = \frac{(2^N)}{f_{\text{clk}} / \text{Prescaler}}$$

---

## Task 2: Generate PWM in Normal Mode (No PWM/CTC Modes!)

**Objective:** Use **Normal Timer Mode** with **Overflow + Compare Match interrupts** to generate a PWM signal with adjustable duty cycle and frequency.

### Requirements:

- Configure a timer in **Normal Mode**.
- In the **Overflow Interrupt**: set/toggle the OCnX pin (start of cycle).
- In the **Compare Match Interrupt**: clear/toggle the OCnX pin (duty cycle point).
- Provide functions:

```
void PWM_init(uint16_t freq, uint8_t duty);  
void PWM_setDuty(uint8_t duty);  
void PWM_setFreq(uint16_t freq);
```

- Show demo on an LED (brightness control + change frequency).

### Hints:

- Datasheet sections:
  - **OCnX pins:** OC0A = PD6, OC0B = PD5, OC1A = PB1, OC1B = PB2, etc.
  - Registers: **OCRnX**, **TIMSKn**, **TCCRnA**, **TCCRnB**.
  - Interrupts: **TOVn** (overflow), **OCFnX** (compare match).
- Frequency formula (since timer resets only on overflow):

$$f_{\text{PWM}} = \frac{f_{\text{clk}}}{N \cdot 2^N}$$

where  $N$  = prescaler.

- Duty cycle:

$$D(\%) = \frac{OCRnX}{2^N} \times 100$$

## Deliverables

1. **.c** and **.h** files for each task.
2. Small README explaining:
  - Which timer and prescaler you used.
  - How you mapped overflow to frequency.
  - How compare match relates to duty cycle.
3. Demos:
  - **Task 1:** Blink LED every 1000 ms with your **millis()**.
  - **Task 2:** Drive LED with PWM → vary brightness + show frequency change.